

AXIA

Complete System Architecture, Data Flow Map, and Observability Stack

Version 1.0 | June 2026
Axia Engineering Team

Executive Summary

This document provides a comprehensive analysis of the Axia platform's data flow architecture, identifies critical issues affecting page loading and reliability, and documents the monitoring infrastructure implemented to ensure observability across the entire application. The investigation revealed systemic problems with loading state management, error handling, and a complete absence of production monitoring capabilities.

Key Findings

- **Projects Page Crash:** React hooks violation caused the page to crash every time it was opened. The usePermissions hook was called inside an IIFE within JSX, violating the Rules of Hooks.
- **Perpetual Loading States:** 12+ pages showed infinite loading spinners because safe-convex-react returned undefined for both loading AND error states, making them indistinguishable.
- **EvidenceExport Blank Page:** Three concurrent Convex queries with no timeout or error fallback meant the page showed a permanent skeleton when any query failed.
- **Dashboard Spinner:** Eight concurrent Convex queries with a single AND-gate loading check meant the page would spin forever if any one query failed.
- **No Monitoring:** Zero error tracking, zero analytics, zero observability. Production errors were invisible.
- **Dark Mode Broken:** @theme inline in Tailwind CSS 4 was baking light-mode hex values directly into utility classes, making dark mode CSS variable overrides completely ineffective.

Fixes Implemented

Fix	Scope	Status
safe-convex-react error state	Foundation	Done
useQueryTimeout hook (5-6s)	All data pages	Done
PageLoader / QueryErrorState	Reusable components	Done
Projects crash (hooks violation)	Projects.tsx	Done
EvidenceExport timeout fallback	EvidenceExport.tsx	Done
Dashboard 8-query timeout	Dashboard.tsx	Done
9 pages: timeout + error fallback	9 page files	Done
Sentry error tracking	Production monitoring	Done

Fix	Scope	Status
PostHog product analytics	User behavior tracking	Done
Dark mode fix (@theme inline)	index.css	Done

System Architecture Overview

Axia is built on a Vite + React 19 + TypeScript + Tailwind CSS 4 stack with Convex as the sole backend for real-time data, authentication, and server functions. The application follows a single-page application architecture with client-side routing via React Router v7. All data flows through Convex's real-time subscription model, where useQuery hooks establish persistent WebSocket connections that automatically update the UI when data changes.

Component Hierarchy

The application is structured as a deeply nested component tree where each layer adds specific capabilities. The root starts with React's StrictMode, then layers on theming, instrumentation, Convex connectivity, authentication, routing, and workspace scoping. This hierarchical design ensures that every child component has access to the full set of contexts it needs.

```
StrictMode +-- VlyToolbar +-- ThemeProvider (dark/light toggle, localStorage
persistence) +-- InstrumentationProvider (Sentry + PostHog init) +--
ConvexProvider (WebSocket real-time connection) +-- ConvexAuthProvider
(sign-in/sign-up/sign-out) +-- BrowserRouter (client-side routing) +--
WorkspaceProvider (active workspace, permissions) +-- ProfileModal (global) +--
Routes +-- Public Routes (Landing, Auth, OwnerDashboard) +-- ProtectedRoute +--
DashboardLayout +-- CollapsibleSidebar +-- Outlet (page content)
```

Technology Stack

Layer	Technology	Purpose
Frontend	React 19 + TypeScript	UI framework with type safety
Build	Vite 6 + SWC	Fast HMR, optimized production builds
Styling	Tailwind CSS 4 + @theme	Utility-first CSS with design tokens
Routing	React Router v7	Client-side SPA routing with guards
Backend	Convex	Real-time database, functions, auth
Auth	@convex-dev/auth	Password, Google, GitHub, Email OTP
State	Convex useQuery/useMutation	Real-time reactive data subscriptions
Monitoring	Sentry + PostHog	Error tracking + product analytics
Animation	Framer Motion	Page transitions and micro-interactions

Data Flow Architecture

Data flows through the Axia application via six distinct patterns, each designed to handle different connectivity and authentication scenarios. Understanding these patterns is critical for diagnosing loading issues and planning new features. All data originates from Convex's real-time subscription model, with fallback mechanisms for offline, unauthenticated, and error scenarios.

Pattern A: Direct Convex

The most common pattern. A page component calls `useQuery(api.module.function, args)` which establishes a real-time WebSocket subscription. Convex automatically pushes updates to the UI whenever the underlying data changes. This pattern is used by the majority of data-fetching pages including Clients, Invoices, Pipeline, Proposals, Tags, Goals, and Reports. The subscription remains active as long as the component is mounted, and Convex handles reconnection automatically.

Pattern B: Safe Convex with Skip

For APIs that may not be deployed yet (e.g., messaging, teams), pages use the `hasApi` check pattern. If the API exists, the query runs normally; if not, the query is skipped by passing "skip" as the arguments. This prevents runtime crashes from missing functions while maintaining React hooks ordering. Used by Messages, TeamManagement, and the permissions system.

Pattern C: Demo Mode Fallback

Pages that need to work without authentication (e.g., for demo purposes) check `isAuthenticated` and fall back to static mock data when the user is not logged in. This ensures the UI always shows meaningful content. Used by Clients, TimeTracking, Messages, and ClientWorkspace.

Pattern D: Workspace-Scoped Queries

Almost all Convex queries accept an optional `workspaceId` parameter for multi-tenancy. The `WorkspaceProvider` context resolves the active workspace ID and sets `isConvexConnected` accordingly. Pages gate their queries on this flag: if the workspace is not connected, queries are skipped to prevent errors. This creates a cascading dependency where a failing workspace query blocks all downstream data.

Pattern E: Client Token Access

Client-facing pages (ClientWorkspace) use token-based access instead of user authentication. A URL token is validated via `validateWorkspaceToken`, and if valid, the client can view projects, proposals, invoices, and team members. Demo tokens bypass Convex entirely and use

static mock data, allowing immediate preview without backend connectivity.

Pattern F: Subscription Tier Gating

Feature access is gated by subscription tier, stored in `localStorage` as `axia_subscription_tier`. The `useSubscriptionTier` hook reads this value and provides `hasTierAccess(requiredTier)` for checking if premium features should be shown. Tier changes are client-local only and do not sync with Convex, meaning the tier state is per-browser.

Cross-Page Data Sharing

Several Convex query functions serve multiple pages simultaneously. This sharing means that when one subscription is active, the data is cached and instantly available to other components on the same page. However, it also means that a failure in one shared query can cascade across multiple pages.

Convex Function	Pages Using It
<code>clients.crud.getClients</code>	Clients, EvidenceExport
<code>clients.crud.getClientsEnriched</code>	Dashboard, PaymentPatterns
<code>billing.crud.getInvoices</code>	Dashboard, Invoices, Subscription, PaymentPatterns
<code>billing.crud.getInvoiceStats</code>	Dashboard, Invoices, Subscription, PaymentPatterns
<code>pipeline.crud.getDeals</code>	Dashboard, Pipeline
<code>pipeline.crud.getPipelineStats</code>	Dashboard, Pipeline
<code>proposals.crud.getProposals</code>	Dashboard, Proposals
<code>proposals.crud.getProposalStats</code>	Dashboard, Proposals
<code>scope.crud.getScopeDefinitions</code>	Dashboard, EvidenceExport, Scope
<code>evidence.library.getEvidenceLibraryData</code>	EvidenceLibrary, EvidenceExport
<code>tags.crud.getTags</code>	Tags, TeamManagement

Page-by-Page Data Dependencies

The following tables provide a complete inventory of every page in the application, organized by their data dependency complexity and loading state handling quality. Pages are classified into three tiers: Safe (no Convex dependencies), Good (proper loading/error/empty states), and Fixed (previously problematic, now repaired with timeout fallbacks).

Pages Without Convex Dependencies (Safe)

Page	Route	Data Source
Landing	/	useAuth only
Auth	/auth	useAuth + ConvexAuth
HelpCenter	/help-center	Static content
NotFound	*	None
OnboardingSource	Not routed	localStorage
OnboardingUserInfo	Not routed	localStorage
ApiSettings	Not routed	Local/mock

Pages With Proper Loading Handling (Good)

Page	Convex Queries	Timeout	Empty State
Clients	getClients	2s	Yes
Projects	getMyProjects	2s	Yes
PlatformIntegrations	getPlatformConnectionStatus	No	Yes
Messages	listChannels, listMessages	Mock fallback	Yes
TeamManagement	getMembers, getWorkspaceStats	No	Yes
TimeTracking	getCurrentSession, getSessions	No	Yes

Fixed Pages (Previously Stuck Loading)

Page	Problem	Fix Applied
Dashboard	8 queries, AND gate, no timeout	6s timeout + placeholder cards
EvidenceExport	3 queries, no timeout/error	6s timeout + error fallback UI
PaymentPatterns	3 queries, no timeout	6s timeout + retry button
Pipeline	3 queries, no timeout	6s timeout + retry button

Page	Problem	Fix Applied
Proposals	No loading state at all	Added skeleton + 6s timeout
Invoices	Manual timeout pattern	Replaced with useQueryTimeout
Reports	Manual timeout pattern	Replaced with useQueryTimeout
Scope	3 loading checks, no timeout	6s timeout on each section
Subscription	2 loading states, no timeout	6s timeout on tier + billing
Tags	Manual timeout pattern	Replaced with useQueryTimeout
Goals	Manual timeout pattern	Replaced with useQueryTimeout

Authentication & Authorization Flow

Axia uses a multi-layered authentication and authorization system. The primary auth system is built on `@convex-dev/auth`, which supports four sign-in methods: password, Google OAuth, GitHub OAuth, and email OTP. All protected routes are wrapped in a `ProtectedRoute` component that checks authentication status and redirects unauthenticated users to the `/auth` page with a return URL parameter.

Sign-In Methods

Method	Flow	Storage
Password	<code>signIn(signUp: true)</code> with <code>FormData</code>	Convex session cookie
Google OAuth	<code>signIn('google')</code> redirect	Convex session cookie
GitHub OAuth	<code>signIn('github')</code> redirect	Convex session cookie
Email OTP	<code>sendCode()</code> then <code>verifyCode()</code>	Convex session cookie

Authorization Layers

Authorization operates at three levels. First, the **Workspace Level** determines what a user can do within a workspace based on their role (owner, manager, member). The `useWorkspacePermissions` hook provides boolean flags like `canDeleteRecords` and `canShareRecords`. Second, the **Record Level** uses the `usePermissions` hook to check access to specific records based on sharing entries, team membership, and workspace role. Third, the **Subscription Tier** gates premium features based on the user's plan level, stored in `localStorage`.

Client Authentication (Parallel System)

Client-facing pages use a separate authentication system. Instead of Convex auth, clients are identified by their email stored in `localStorage` as `axia_client_email`. The `ClientWorkspace` page uses token-based access where a URL token is validated via `validateWorkspaceToken`. Demo tokens (prefixed with 'demo-') bypass Convex entirely and show static mock data. This parallel system allows external clients to view their project data without creating a full user account.

Convex Backend & Schema

The Convex backend serves as Axia's sole data layer, providing real-time subscriptions, mutations, and actions. The schema is organized into modular table groups that map to different feature domains. All queries are reactive: when underlying data changes, subscribed components automatically receive updates via WebSocket, requiring no manual refresh or polling.

Schema Module Map

Module	Key Tables	Pages Served
authTables	authAccounts, authSessions	Auth
users	users	All protected pages
workspaceTables	workspaces, workspaceMembers, workspaceInvitations	TeamManagement, usePermissions
teamTables	teams, teamMemberships	TeamManagement
projectTables	clients, projects, clientCompanies	Clients, Projects, ClientDashboard
evidenceTables	evidenceSessions, evidenceEvents, evidenceMetadata	EvidenceLibrary, EvidenceExport
pipelineTables	pipelineStages, deals	Pipeline, Dashboard
proposalTables	proposals, proposalTemplates, proposalFollowUps	Proposals, ProposalBuilder
billingTables	invoices, invoiceWorkLinks, paymentReminders	Invoices, InvoiceBuilder, PaymentPatterns
scopeTables	scopeDefinitions, scopeChangeOrders	Scope, Dashboard
messagingTables	channels, channelMembers, messages, reactions	Messages
tagTables	tags	Tags, TeamManagement
goalTables	goals	Goals
featureTables	disputeReports, platformConnections, waitlistEntries	Reports, PlatformIntegrations

safe-convex-react Design

The safe-convex-react module wraps all Convex hooks to prevent runtime errors from crashing the React tree. The design follows three key principles: (1) Errors return safe defaults instead of throwing, (2) Hooks are always called in the same order using Convex's 'skip' mechanism, and (3) The new useQueryResult API returns typed QueryResult objects that distinguish between loading, error, and success states. The useQueryTimeout hook provides a 5-6 second timeout that transitions pages from loading skeletons to error states, eliminating perpetual spinners.

Loading State Architecture

The loading state architecture was the single most impactful area of the audit. The original system had a fundamental design flaw: `safe-convex-react` returned `undefined` for both 'still loading' and 'query errored' states, making them indistinguishable. Pages that checked `data === undefined` to show loading spinners would spin forever on errors, because there was no way to detect that a query had permanently failed.

The Problem: Indistinguishable States

```
// BEFORE: Both loading and error return undefined
const data = useQuery(api.clients.getClients, {});
// data === undefined → Still loading? OR query failed?
// No way to tell!
if (data === undefined) return; // Spins FOREVER on error
```

The Solution: `useQueryTimeout` + `useQueryResult`

```
// AFTER: Timeout breaks the infinite loading loop
const data = useQuery(api.clients.getClients, {});
const isLoading = data === undefined;
const timedOut = useQueryTimeout(isLoading, 6000);
if (isLoading && !timedOut) return;
if (isLoading && timedOut) return; // Data loaded successfully
return;
```

The `useQueryTimeout` Hook

The `useQueryTimeout` hook accepts a boolean `isLoading` flag and a timeout duration (default 5000ms). It returns `true` once the timeout expires while still loading, giving pages a clear signal to transition from the loading skeleton to an error or retry state. The hook automatically resets when loading completes, so it works correctly with Convex's reconnection behavior.

Component Library

Three reusable components were created in `QueryState.tsx`: **PageLoader** combines loading skeleton, timeout error, and empty state into a single wrapper. **QueryErrorState** shows an alert icon, error message, and retry button. **EmptyState** shows an inbox icon, message, and optional action button. These components ensure a consistent user experience across all pages.

Sentry Error Tracking

Sentry provides real-time error tracking, performance monitoring, and session replay for production debugging. The integration is designed to be zero-configuration in development (Sentry silently skips initialization if no DSN is provided) and fully featured in production.

Configuration

Setting	Development	Production
Traces Sample Rate	1.0 (100%)	0.2 (20%)
Replays Session Rate	0 (disabled)	0.1 (10%)
Replays On Error Rate	1.0 (100%)	1.0 (100%)
Source Maps	Generated, not uploaded	Generated + uploaded to Sentry
Error Filtering	Skip ResizeObserver, network, Convex auth	Same + browser extension noise

Integration Points

- **InstrumentationProvider:** Initializes Sentry on app mount, captures unhandled errors and promise rejections, reports to both Sentry and the legacy Vly error system.
- **safe-convex-react:** Reports every Convex query error to Sentry via `reportQueryError()`, including the query name, error message, and arguments.
- **ConvexErrorBoundary:** The top-level React error boundary catches rendering crashes and reports them to Sentry before showing the error banner.
- **ProtectedRoute:** Sets the Sentry user identity (ID, email, name, tier) when a user authenticates, and clears it on sign-out.
- **Vite Plugin:** Uploads source maps to Sentry during production builds for stack trace deobfuscation.

Environment Variables

```
VITE_SENTRY_DSN= # Sentry project DSN (required for production) VITE_SENTRY_ORG= #  
Sentry organization slug VITE_SENTRY_PROJECT= # Sentry project name (default:  
'axia') SENTRY_AUTH_TOKEN= # Auth token for source map upload (CI only)  
VITE_GIT_SHA= # Git commit SHA for release tracking
```

PostHog Product Analytics

PostHog provides product analytics, feature flags, and user behavior tracking. Like Sentry, it gracefully degrades in development when no API key is configured, logging events to the console instead. In production, it captures page views, user actions, and funnel data.

Event Catalog

Event	Trigger	Properties
auth_sign_in	User signs in	method (password/google/github/otp)
auth_sign_up	User creates account	method
auth_sign_out	User signs out	session_duration
page_view	Route change	path, search
sidebar_toggle	Sidebar expand/collapse	is_expanded
theme_toggle	Dark/light switch	from, to
client_created	New client added	source (manual/import)
invoice_created	New invoice created	amount, client_id
invoice_sent	Invoice sent to client	invoice_id, format
proposal_created	New proposal created	deal_id, value
deal_created	New deal in pipeline	stage, value
deal_moved	Deal moved between stages	from_stage, to_stage
evidence_export	Evidence exported	format, item_count
upgrade_clicked	Upgrade CTA clicked	current_tier, target_tier
convex_query_error	Convex query fails	queryName, error
page_load_timeout	Page exceeds 6s loading	pageName, errorMessage

Feature Flags

PostHog feature flags are accessible via the `isFeatureEnabled()` and `getFeatureFlagVariant()` functions in `monitoring.ts`. These allow gradual feature rollouts and A/B testing without code deployments. Flags fall back to default values when PostHog is not initialized.

Environment Variables

```
VITE_POSTHOG_KEY= # PostHog project API key VITE_POSTHOG_HOST= # PostHog host
(default: https://us.i.posthog.com) VITE_GIT_SHA= # Git SHA for release tracking
```

Issues Found & Fixes Applied

This section documents every issue discovered during the architecture audit and the specific fix applied to each one. Issues are ordered by severity and user impact.

#	Issue	Root Cause	Fix
1	Projects page crash	usePermissions called in IIFE (hooks violation)	Moved hook to top level, removed IIFE
2	EvidenceExport blank	No timeout/error state on 3 queries	Added 6s timeout + error fallback UI
3	Dashboard perpetual spinner	8-query AND gate, no timeout	Added 6s timeout + placeholder cards
4	safe-convex-react erases errors	Both loading and error return undefined	Added useQueryResult typed API + useQueryTimeout
5	9 pages stuck loading forever	No timeout on useQuery results	Added useQueryTimeout(5000-6000) to all
6	Zero monitoring/observability	No error tracking or analytics	Implemented Sentry + PostHog integration
7	Dark mode broken	@theme inline bakes light-mode values	Changed to @theme (CSS variable references)
8	Proposals hooks violation	usePermissions in JSX callback	Removed hook call, used workspace-level permission

Detailed Fix: Projects Page Crash

The Projects page crashed because `usePermissions` was called inside an IIFE within JSX. This violated React's Rules of Hooks in two ways: (1) the hook was inside a conditional (`selectedProjectId &&`), so it was skipped on the first render, and (2) it was inside a nested function, not at the top level of the component. When the auto-select `useEffect` set `selectedProjectId`, the hook suddenly appeared on the next render, adding 5 internal hooks (`useWorkspaceContext` + 2x `useQuery` + 2x `useMemo`) and changing the hook count. React detected this mismatch and crashed. The fix moved `usePermissions` to the top level, where it always runs in the same order. When the selected project is null, the hook gracefully returns `NO_ACCESS` with all boolean flags set to false.

Environment Variables & Deployment

Required Environment Variables

Variable	Required	Description
VITE_CONVEX_URL	Yes	Convex deployment URL (falls back to hardcoded)
VITE_SENTRY_DSN	Production	Sentry project DSN for error tracking
VITE_SENTRY_ORG	CI	Sentry organization slug for source maps
VITE_SENTRY_PROJECT	CI	Sentry project name (default: axia)
SENTRY_AUTH_TOKEN	CI	Sentry auth token for source map upload
VITE_POSTHOG_KEY	Production	PostHog project API key for analytics
VITE_POSTHOG_HOST	Optional	PostHog host URL (default: us.i.posthog.com)
VITE_GIT_SHA	Recommended	Git SHA for release tracking in Sentry/PostHog
VITE_VLY_APP_ID	Optional	Vly editor integration ID

Build & Deploy

```
# Development (no monitoring keys needed) npx vite build # Production build with
source maps VITE_GIT_SHA=$(git rev-parse HEAD) \ SENTRY_AUTH_TOKEN=your-token \
VITE_SENTRY_DSN=https://xxx@sentry.io/xxx \ VITE_POSTHOG_KEY=phc_xxx \ npx vite
build # Preview locally cd dist && python3 -m http.server 3000 --bind 0.0.0.0
```

Sentry Source Map Upload

The `@sentry/vite-plugin` is configured in `vite.config.ts` to automatically upload source maps to Sentry during production builds. This requires the `SENTRY_AUTH_TOKEN` environment variable to be set in CI. The plugin is disabled in development and when no auth token is provided, so it does not interfere with local development. After upload, source map files are deleted from the `dist` directory to prevent them from being served publicly.

PostHog Feature Flags

Feature flags can be configured in the PostHog dashboard and accessed via the `isFeatureEnabled()` function. Flags are evaluated server-side and cached locally. When PostHog is not configured, flags return their default values. This allows safe use of feature flags in code without worrying about the analytics service being unavailable.